

Systeme d'Exploitation

Pr. Mohammed AIT DAOUD

Programme système

CHAPITRE 0 : Introduction générale

- **Partie 1 : Fondamentaux du Systèmes d'Exploitation**

- CHAPITRE 1 : Qu'est ce qu'un système d'exploitation ?
- CHAPITRE 2 : Introduction aux système d'exploitation Linux
- CHAPITRE 3 : Système d'exploitation UBUNTU

- **Partie 2 : Les Commandes de Base du Système**

- CHAPITRE 4 : Gestion des fichiers et des répertoires
- CHAPITRE 5 : Redirection des Entrées/Sorties
- CHAPITRE 6 : Gestion des utilisateurs et des groupes

- **Partie 3 : Langage de Programmation**

- CHAPITRE 7 : Shell, c'est quoi exactement
- CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL
- CHAPITRE 9 : Langage de manipulation et de traitement de texte AWK

- **Partie 4 : Projet pratique**

PARTIE 3 : LA PROGRAMMATION SHELL

CHAPITRE 7 : Shell, c'est quoi exactement

- Objectifs :
 - Comprendre le rôle du Shell dans un système Linux.
 - Découvrir les différents types de Shell.
- Contenu :
 - Définition et rôle du Shell.
 - Différents types de Shell : Bash, Zsh, Ksh, Fish.
 - Fonctionnement d'un interpréteur de commandes.
- Activités :
 - Identification du Shell utilisé (echo \$SHELL).
 - Expérimentation avec différentes commandes Shell.



CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

- Il s'agit d'une interface texte entre l'utilisateur et le système
 - ◆ Tout se fait au clavier
 - ◆ Pas de clic de souris
- L'utilisateur tape des commandes qui sont exécutées par le système
 - ◆ Le shell est donc un « interpréteur de commandes »
 - ◆ Chaque commande a une syntaxe particulière
 - ◆ Il existe des milliers de commandes différentes
- Les commandes peuvent aussi provenir d'un fichier
 - ◆ Le fichier contient les commandes à exécuter
 - ◆ L'utilisateur appelle le fichier plutôt que de taper toutes les commandes
 - Utile pour les tâches répétitives
- Le shell reste le moyen le plus efficace pour contrôler le système
 - ◆ C'est aussi le plus utilisé sous Linux/Unix

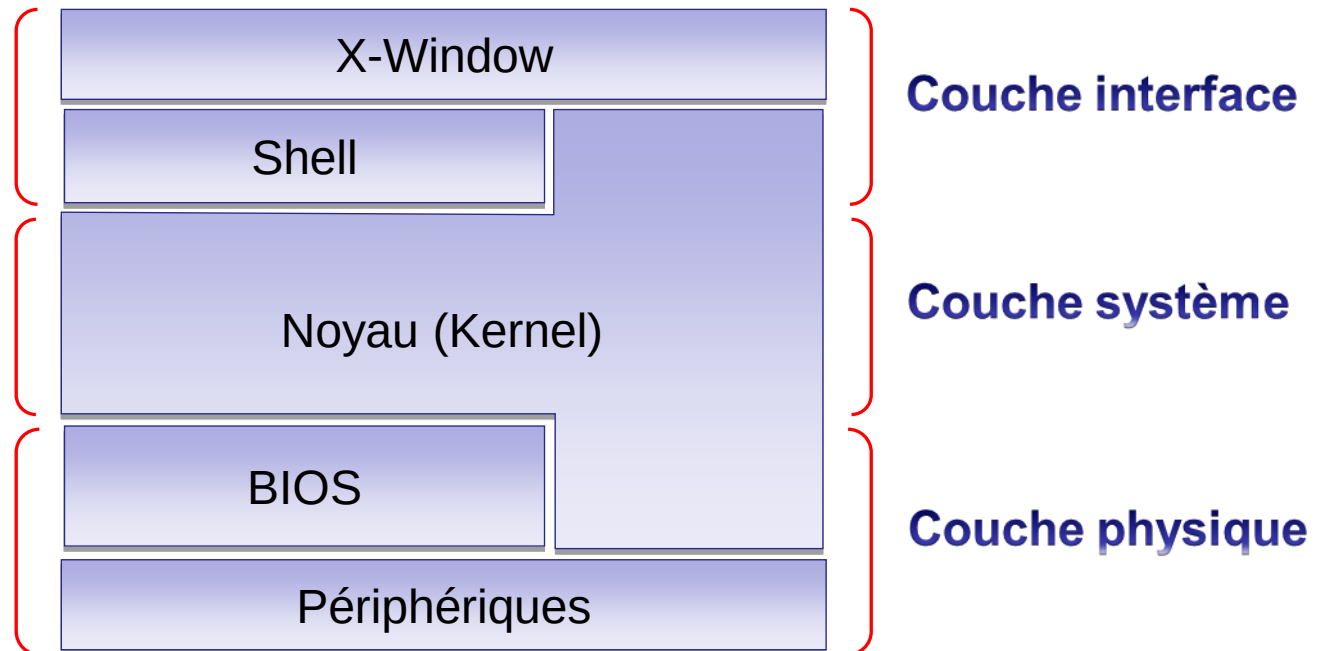
DÉFINITION

- Un shell est la liaison la plus élémentaire entre l'utilisateur et le système d'exploitation, c'est à dire le programme de gestion de la ligne de commande.

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Architecture GNU/linux

- Divisée en 3 couches distinctes
 - ◆ La couche physique : Périphériques et BIOS
 - ◆ La couche système : Gérée par le noyau
 - ◆ La couche interface : le Shell et/ou le système X-Window

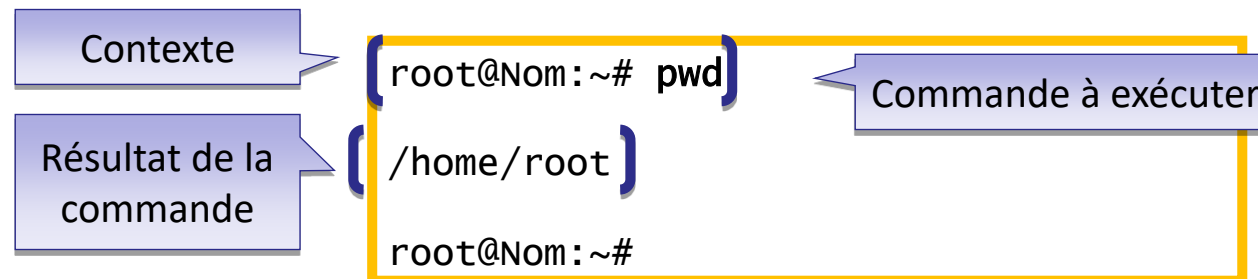


X-Window : Interface graphique

- C'est l'environnement graphique de tous les systèmes Unix et Linux
- Basée sur la relation client-serveur
 - ◆ Le serveur X peut accepter un client distant afin de déporter l'affichage sur une autre machine
- Pour Linux, le serveur X se nomme XFree86
- Le serveur X fonctionne avec un « gestionnaire de fenêtrage »
 - ◆ Il en existe plusieurs (Kde, Gnome, ...)
 - ◆ Définit l'aspect du bureau, des fenêtres et des menus
- Exemple avec Gnome

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

- Le shell est un **interpréteur de commandes**
 - ◆ Permet à l'utilisateur d'interagir avec le système
 - ◆ Il lit et exécute les commandes de l'utilisateur
- Il en existe plusieurs
 - ◆ Le shell « bash » est le plus courant
 - ◆ Mais aussi les shells « csh », « ksh », « tcsh »
- C'est aussi un véritable **langage de programmation**
 - ◆ Il sera possible d'écrire des scripts exécutant des commandes répétitives



CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Les différents types de shell

- Il existe plusieurs types de shell
 - ◆ **sh** : Bourne Shell. L'ancêtre de tous les shells.
 - ◆ **bash** : Bourne Again Shell. Une amélioration du Bourne Shell, disponible par défaut sous Linux et Mac OS X.
 - ◆ **ksh** : Korn Shell. Un shell puissant assez présent sur les Unix propriétaires, mais aussi disponible en version libre, compatible avec bash.
 - ◆ **csh** : C Shell. Un shell utilisant une syntaxe proche du langage C.
 - ◆ **tcsh** : Tenex C Shell. Amélioration du C Shell.
 - ◆ **zsh** : Z Shell. Shell assez récent reprenant les meilleures idées de bash, ksh et tcsh.
 - ◆ L'interpréteur de commande MS-DOS (Sous Windows)
 - ◆ PowerShell (Windows 2008 server)
- Sous Linux, on peut afficher son shell
 - `echo $SHELL`
 - `Ps`
 - `bash --version`

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ? C

Présentation de la ligne de commande (CLI)

- La ligne de commande se présente sous forme de texte ayant la signification suivante :

Utilisateur courant Répertoire courant

root@Nom:/home/mohamed# |

Nom de la machine # : Superutilisateur
\$: Utilisateur normal

- Il suffit alors de taper une commande pour qu'elle soit exécutée
- Une commande peut être appelée :
 - En tapant son nom puis des arguments ou paramètres
 - Exemple permettant d'afficher les commandes internes et externes

```
root@Nom:/home/mohamed# type -t sleep
```

Ex: `$ type -t sleep`

file => *sleep* est une commande externe

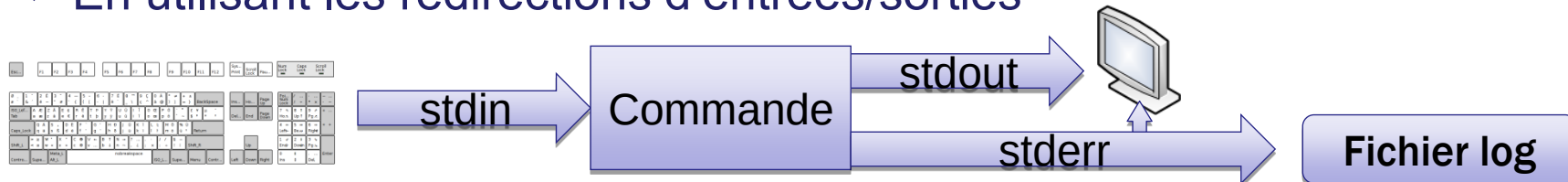
`$ type -t echo`

builtin => *echo* est une commande interne du shell

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Les entrées/sorties

- Chaque commande dispose d'entrées/sorties :
 - ◆ Comme l'écran (sortie) ou le clavier (entrée)
- 3 types différents:
 - ◆ L'entrée standard définie par le symbole « **stdin** » et le descripteur **0**
 - Provenant du clavier par défaut
 - ◆ La sortie standard « **stdout** » et le descripteur **1**
 - Dirigée vers l'écran par défaut
 - ◆ La sortie d'erreurs « **stderr** » et le descripteur **2**
 - Egalement redirigée vers l'écran par défaut
- Possibilité de modifier le comportement par défaut
 - ◆ En utilisant les redirections d'entrées/sorties



CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Redirections des entrées/sorties

- Capacité de rediriger les entrées/sorties d'une commande
 - ◆ « stdout » ou « stderr » vers un fichier plutôt qu'à l'écran
 - ◆ « stdin » depuis un fichier plutôt que le clavier
- Utilisation des opérateurs suivants :
 - ◆ > : Redirection de la sortie vers un fichier
 - ◆ >> : Redirection de la sortie à la fin du fichier (concaténation)
 - ◆ < : Redirection de l'entrée depuis un fichier
- Exemple de redirection de la sortie vers un fichier :

```
ls -l /etc > listing-etc.txt
```

- ◆ Le listing est écrit dans le fichier « listing-etc.txt »

« /etc » tapé
au clavier

ls

listing-etc.txt

- Redirection de l'entrée de la commande « wc » depuis un fichier
 - ◆ Compte le nb de lignes du fichier « listing-etc.txt »

```
wc -l < listing-etc.txt
```

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Redirections (2)

- 2 syntaxes supplémentaires possibles pour les redirections

```
n>&m ou  
n>fichier
```

- ◆ n : Numéro du descripteur à rediriger
 - ◆ m : Numéro du descripteur vers lequel on va rediriger
 - ◆ fichier : Fichier vers lequel s'effectuera la redirection
- Rappel : 0 (entrée standard), 1 (sortie standard), 2 (sortie erreur)
 - Exemple : Rediriger la sortie d'erreur vers un fichier
 - ◆ Les messages d'erreurs seront écrits dans « erreurs.log »

```
cp /home/karim /home/salim 2>erreurs.log
```

- Exemple : Rediriger la sortie standard vers la sortie d'erreur

```
cp /home/karim /home/salim 1>&2
```

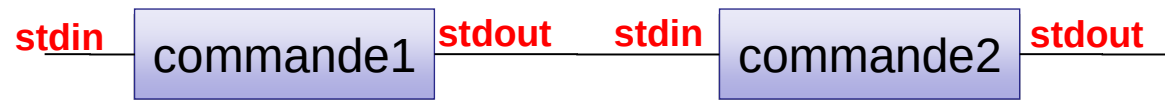
- Exemple : Rediriger stdout ET stderr vers un fichier

```
commande > fichier.txt 2>&1
```

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Les tubes (pipes)

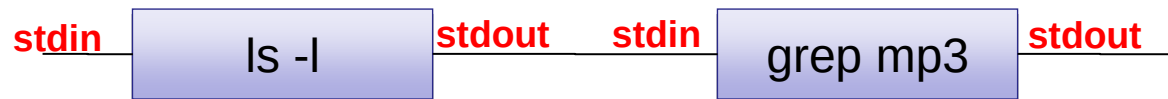
- Il s'agit de rediriger la sortie d'une commande vers l'entrée d'une autre avec l'opérateur « | »



- Élaborer des commandes complexes en une seule ligne
 - ◆ Exemple : Filtrer le résultat de la commande « ls » avec « grep »

```
ls -l /etc | grep 'mp3'
```

- On obtient la liste des fichiers contenant « mp3 »



```
rout@fredon:~$ ls -l /etc | grep 'mp3'
-rw-r--r-- 1 karim karim 0 2008-08-27 15:16 morceau1.mp3
-rw-r--r-- 1 karim karim 0 2008-08-27 15:16 morceau2.mp3
-rw-r--r-- 1 karim karim 0 2008-08-27 15:16 morceau3.mp3
-rw-r--r-- 1 karim karim 0 2008-08-27 15:16 morceau4.mp3
```

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Exécution conditionnelle de commandes

- Pour exécuter une seule commande, rien de plus simple
 - ◆ Taper son nom au clavier
- Pour exécuter plusieurs commandes à la suite
 - ◆ L'exemple suivant crée un répertoire, s'y déplace et crée un fichier

```
mkdir toto ; cd toto ; touch alire.txt
```
- L'exécution conditionnelle de commandes est possible
 - ◆ Les commandes s'exécutent les unes après les autres sous condition
 - ◆ Utilisation des opérateurs && et ||
 - L'exemple suivant exécute commande1 et commande2 seulement si le résultat renvoyé par commande1 est égal à 0

```
commande1 && commande2
```
 - Même chose mais si le résultat de commande1 est différent de 0

```
commande1 || commande2
```

CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

Ca marche ou ca marche pas ?

- Une commande renvoie toujours une valeur
 - ◆ 0 si la commande s'est exécutée correctement
 - ◆ 1 ou différent de zéro dans le cas contraire
 - Exemple
 - ◆ La variable « \$? » correspond à la valeur renvoyée par la dernière commande exécutée (Donc la commande « cd »)
- ```
cd toto
Bash : cd toto : Le répertoire n'existe pas
echo $?
1
```
- Cette valeur peut-être exploitée dans un script pour connaître le résultat d'une commande avant d'exécuter la suite



# CHAPITRE 7 : SHELL, C'EST QUOI EXACTEMENT ?

## Exercices

- Copier le fichier « /etc/syslog.conf » vers son répertoire home

```
cp /etc/syslog.conf ~/
```

- A l'aide de la commande « cat », filtrer les lignes contenant le mot « internet » dans le fichier « toto.txt »

```
cat toto.txt | grep 'internet'
```

- Renommer le répertoire « rapports » vers « rapport2007 » et stocker les messages d'erreurs dans un fichier « mv-rapport.log »

```
mv rapport/ rapports2007 2>mv-rapport.log
```

- Créer le fichier « rap2009 » dans le répertoire « rapports » seulement s'il existe

```
test -e rapports && touche rapports/rap2009
```

# PARTIE 3 : LA PROGRAMMATION SHELL

## CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

- Objectifs :
  - Apprendre à écrire des scripts Shell.
  - Automatiser des tâches système.
- Contenu :
  - Syntaxe de base d'un script Shell.
  - Variables et arguments (\$1, \$2, \$@).
  - Structures de contrôle (if, for, while, case).
  - Gestion des fichiers et processus dans un script (ls, rm, kill).
- Activités :
  - Création d'un script Shell simple.
  - Automatisation d'une tâche répétitive (exemple : sauvegarde de fichiers).



## Modes d'exécution d'une commande

- Exécution séquentielle (Par défaut)
  - ♦ le shell lit la commande entrée par l'utilisateur, l'analyse, la prétraite et si elle est syntaxiquement correcte, l'exécute.
- Exécution en arrière-plan
  - ♦ L'exécution en arrière-plan permet à un utilisateur de lancer une commande et de récupérer immédiatement la main pour lancer « en parallèle » la commande suivante sur terminal (parallélisme logique) .
  - ♦ On utilise le caractère & pour lancer une commande en arrière-plan
  - ♦ Ex : `$ sleep 5 &`
  - ♦ => Ici, sleep 5 s'exécute en arrière-plan et le terminal reste accessible immédiatement.

## CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

### Commentaires

- Un commentaire débute avec le caractère **#** et se termine avec la fin de la ligne.
- Un commentaire est ignoré par le shell.

```
Ex: $ echo bonjour # l'ami
 bonjour
 $ echo coucou #l' ami
 coucou
 $ # echo coucou
 $
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Fichiers shell

- Lorsqu'un traitement nécessite l'exécution de plusieurs commandes, il est préférable de les sauvegarder dans un fichier plutôt que de les retaper au clavier chaque fois que le traitement doit être lancé.
- Ce type de fichier est appelé fichier de commandes ou fichier shell ou encore **script shell**.
- Un script est un fichier contenant un ensemble de commandes exécutées séquentiellement
  - ◆ Sous forme de fichier texte contenant les commandes
- Le script ne peut être exécuté que par un interpréteur
  - ◆ « /bin/sh » pour le shell bash
- Le langage de script shell est un langage évolué offrant de nombreuses possibilités
  - ◆ Boucles, variables, tests avec if, création de fonctions, ...
- Dans quels cas utilise-t-on les scripts ?
  - Pour effectuer un travail répétitif, Pour des tâches d'administration système, Pour installer des programmes, Au démarrage du système pour démarrer les services et applis

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Structure et exécution d'un script

- Par convention les **shell scripts** se terminent généralement (pas obligatoirement) par « **.sh** » pour le **Bourne Shell** et le **Bourne Again Shell**, par « **.ksh** » pour le **Korn Shell** et par « **.csh** » pour le **C Shell**.
- Pour lancer l'exécution d'un **fichier shell**, on peut utiliser la commande : **bash**
  - **\$ bash monscript**
- Il est plus simple de lancer l'exécution d'un programme shell en tapant directement son nom, comme on le ferait pour une commande unix ou une commande interne.
- Pour que cela soit réalisable, deux conditions doivent être remplies :
  - l'utilisateur doit posséder les permissions r (lecture) et x (exécution) sur le fichier shell :
    - **\$ chmod u+x monscript**
    - Pour l'exécuter : **\$ ./monscript**
  - Pour éviter le ./ le répertoire dans lequel se trouve le fichier shell doit être présent dans la liste des chemins contenue dans **PATH**. :
    - **\$ PATH=\$PATH:.**
    - **\$ monscript**

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Exécuter un script

- Pour exécuter le script, il faut appeler l'interpréteur

```
root@Nom:~# sh monscript
```

- Possibilité de simplifier l'appel en ajoutant la ligne suivante en tête du script

```
#!/bin/bash
```

- ◆ L'appel est alors plus simple

```
root@Nom:~# ./monscript
```

- L'utilisateur courant doit posséder le droit « x » pour le fichier

- ◆ Exemple : Seul, l'utilisateur « karim » pourra exécuter le script

```
-rwxr--r-- 1 aziz fsbm 7406 2008-08-15 14:44 script.sh
```

- ◆ Pour autoriser les membres du groupe « fsbm »

```
chmod g+x script.sh
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Exercice 1 :

- 1.) A l'aide d'un éditeur de texte, créer un fichier **monscript** contenant les lignes suivantes :

```
#!/bin/bash
premier
echo -n "La date du jour est: " date
```

- La notation **#!** en première ligne d'un fichier interprété précise au shell courant quel interpréteur doit être utilisé pour exécuter le programme (dans cet exemple, il s'agit de /bin/bash).
- La deuxième ligne est un commentaire.
- 2.) Vérifier le contenu de ce fichier.
- 3.) Pour lancer l'exécution d'un fichier shell ***monscript***, on peut utiliser la commande : ***bash monscript*** ou ***./monscript*** ou directement ***monscript***

Ex :

```
$ monscript
la date du jour est : jeudi 27 février 2014, 17:34:25 (UTC+0100)
$
```



# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Exercice 2:

- Écrire un programme shell monscript1 qui affiche le nom de connexion de l'utilisateur et le chemin absolu de son répertoire courant de la manière suivante :

- Ex :

```
$./monscript1
mon nom de connexion est : stagiaire
mon repertoire courant est : /home/stagiaire
$
```

- Correction :

```
#!/bin/bash

Affichage des informations
echo "mon nom de connexion est : WHOAMI"
echo "mon repertoire courant est : Pwd"
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Les types de parameters

- Le shell distingue trois types de paramètres :

- les ***variables***, identifiées par un *nom*

- ◆ **Ex : a , PATH**

a=10 → Affectation d'une valeur à une variable.  
echo \$a → Affichage du contenu de la variable a.  
PATH → Variable système contenant les chemins des exécutable.

- les ***paramètres de position***, identifiés par un *numéro*

- ◆ **Ex : 0 , 1 , 12**

./script.sh arg1 arg2 arg3  
\$0 → Nom du script (script.sh)  
\$1 → arg1  
\$2 → arg2  
\$3 → arg3

- les ***paramètres spéciaux***, identifiés par un *caractère spécial*

- ◆ **Ex : # , ? , \$**

| Paramètre | Description                                                                  |
|-----------|------------------------------------------------------------------------------|
| \$#       | Nombre d'arguments passés au script                                          |
| \$?       | Code de retour de la dernière commande exécutée (0 = succès, autre = erreur) |
| \$\$      | PID du script en cours                                                       |
| \$_       | PID du dernier processus exécuté en arrière-plan                             |
| @         | Liste des arguments sous forme de tableau                                    |
| *         | Liste des arguments sous forme d'une seule chaîne                            |

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Les Variables

- Une variable est identifiée par un nom, Une suite de lettres, de chiffres ou de caractères espace souligné ne commençant pas par un chiffre.
- Les lettres majuscules et minuscules sont différenciées.
- Les variables peuvent être classées en trois groupes :
  - ◆ A- les variables **utilisateur** (ex : a, valeur)
  - ◆ B- les variables **prédéfinies du shell** (ex : **PS1, PATH, REPLY, IFS, HOME**)
  - ◆ C- les variables **prédéfinies de commandes unix** (ex : **TERM**).
- En général, les noms des **variables utilisateur** sont en lettres **minuscules** tandis que les noms des **variables prédéfinies** (du shell ou de commandes unix) sont en **majuscules**.
- Les principales commandes de manipulation des variables sont :
  - ◆ **Echo, Print, Export, read, Set, unset, Shift**

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Les Variables simples

- Nous appellerons variable simple toute variable définie par l'utilisateur pour ses propres besoins.
- Désignation de la variable
  - ♦ Pour désigner une variable, il suffit d'utiliser la syntaxe suivante:  
`variable`
  - ♦ Si la variable doit être suivie d'une valeur autorisée (exemple de nom de variable auquel on ajoute séquentiellement un nombre), la notation est alors la suivante:  
`{variable}nombre`
- Affichage d'une variable
  - ♦ La commande:  
`echo [-n] [texte] $variable`
  - ♦ permet d'afficher la valeur de la variable.
  - ♦ On peut aussi l'intégrer dans du texte.
  - ♦ L'option `-n` permet d'éviter le passage à la ligne après l'affichage

## Types d'affectation des Variables

### ■ Affectation directe

- ◆ Syntaxe : `nom=[valeur] [ nom=[valeur] ... ]`
- ◆ le symbole `=` et la valeur à affecter forment une seule chaîne de caractères.
- ◆ Plusieurs affectations peuvent être présentes dans la même ligne de
- ◆ commande

◆ Ex : `$ x=coucou y=bonjour`

◆=> la variable x contient la chaîne de caractères coucou

◆=> la variable y contient la chaîne bonjour

◆ EX : `$ echo x est $x`  
`x est coucou`  
`$`

- ◆ Lorsque l'on souhaite effectuer une concaténation, il est possible d'utiliser l'opérateur `+=` de bash. EX :

```
$ x+=
$ echo $x
coucoubonjour
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Types d'affectation des Variables

- Affectation par lecture : saisie au clavier

- Syntaxe : `read [ var1 ... ]`

◆ Ex :

```
$ read a b
```

- ◆ bonjour Monsieur AZIZ => chaînes saisies par l'utilisateur et enregistrées respectivement dans les variables a et b

```
$ echo $a
bonjour
$
```

```
$ echo $b
Monsieur AZIZ
$
```

- ◆ Lorsque la commande interne read est utilisée sans argument, la ligne lue est enregistrée dans la variable prédéfinie du shell **REPLY**:

```
$ read
bonjour tout le monde
$echo $REPLY
bonjour tout le monde
$
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Types d'affectation des Variables

- L'option « **-p** » de read affiche une chaîne d'appel avant d'effectuer la lecture

```
read -p chaîne_d_appel [var ...]
```

◆ Ex :

```
$ read -p "Entrez votre prenom : " prenom
Entrez votre prenom : Eric
$ $
echo $prenom
Eric
$
```

- Remarque : s'il y a moins de variables que de mots dans la ligne lue, le shell affecte le 1er mot à la première variable, le 2ème mot à la 2ème variable, etc., la dernière variable reçoit tous les mots restants:

◆ EX :

```
$ read a b c
un bon jour coucou
$ $
echo $a
un
$ echo $c
jour coucou
$
```

## Types d'affectation des Variables

- Affectation par lecture : saisie au clavier

Remarque : s'il y a Plus de variables que de mots dans la ligne lue, chaque variable reçoit un mot et après épuisement de ces derniers, les variables excédentaires sont vides:

◆ EX :

```
$ read a b
un
$ $
echo $a
un
$ $
echo $b
=> valeur null
$
```

- ◆ La commande **read** propose plusieurs options Intéressantes
  - ◆ **-p** : afficher un message de prompt
  - ◆ **-n** : limiter le nombre de caractères
  - ◆ **-s** : ne pas afficher le texte saisi



# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Types d'affectation des Variables

- Ecrire un programme shell deux fois qui affiche le message "Entrez un mot : ", lit le mot saisi par l'utilisateur puis affiche ce mot deux fois sur la même ligne :

- Ex :

```
$ deuxfois
Entrez un mot : toto
toto toto
$
```

- Correction :

```
#!/bin/bash

Demander à l'utilisateur de saisir un mot
echo -n "Entrez un mot : "
read a
Afficher le mot deux fois sur la même ligne
echo "$a $a"
```

```
#!/bin/bash

read -p "Entrez un mot:" a
echo $a $a
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Variable Constante:

- Variable en « lecture seule » ou Constante:

```
declare -r nom=valeur [nom=valeur ...]
```

- Ex :

```
$ declare -r mess=bonjour
$
```

- On dit que la variable **mess** possède l'**attribut r**.
- Une tentative de modification de la valeur d'une constante provoque une erreur.

```
$ mess=salut
bash: mess : variable en lecture seule
$
```

- Pour connaître la liste des constantes définies : **declare -r**

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Variable Environnement:

- Dans un Shell Bash (ou tout autre interpréteur de commandes), les variables peuvent être classées en deux types principaux :.
- Variables locales
  - ◆ Une variable locale est une variable qui **n'est visible que dans le processus où elle a été déclarée.**
  - ◆ **Elle disparaît dès que le processus se termine** et n'est pas accessible aux sous-processus.
  - ◆ Ces variables sont souvent utilisées pour stocker temporairement des informations au sein d'un script ou d'une session Bash.
- Variables d'environnement
  - ◆ Une variable d'environnement est une variable qui est transmise aux sous-processus.
  - ◆ **Elle est héritée par tous les processus fils** lancés à partir du Shell actuel.
  - ◆ Elle est **utilisée pour configurer l'environnement du Shell** et des applications.

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Variable Environnement:

- D'ordinaire, une variable n'est utilisée que dans le shell script où elle reçoit son affectation, si on souhaite l'utiliser pour les programmes appelés ultérieurement, on doit l'exporter.

```
VARIABLE=valeur
export VARIABLE
```

- Permet de rendre global (visible pour des shells secondaires, et non les autres shells) des variables qui, par défaut, sont locales au shell courant.

- Ex :

```
$ ma_variable=toto
$ export ma_variable
bash
$ echo $ma_variable #affiche toto
```

- Une **variable d'environnement** est une variable qui peut être utilisée par les processus enfants du Shell.



## CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

---

- Les deux commandes (declare -x et export) sont équivalentes, mais export est plus couramment utilisé en scripting Bash.
- La variable VARIABLE peut être appelée par un autre script dans le même shell
- Mais reste inconnue pour les autres shell (nouvelle console).
- Ex :

- Les variables Bash ne sont pas typées : À l'inverse de nombreux langages de programmation. Essentiellement, les variables bash sont des chaînes de caractères mais, suivant le contexte, Bash autorise des opérations entières et des comparaisons sur ces variables, le facteur décisif étant la seule présence de chiffres dans la variable.
- `declare -i` permet de déclarer une variable de type entier
- Les commandes intégrées `declare` ou `typeset`, qui sont des synonymes exacts, permettent de modifier les propriétés des variables. Ceci est une forme très faible du typage disponible dans certains langages de programmation. La commande `declare` est spécifique à la version 2 ou ultérieure de Bash. La commande `typeset` fonctionne également dans les scripts ksh.

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Les attributs de variables : TYPESET

- La commande **typeset** est utilisée en KornShell (ksh) et est également disponible dans Bash (bien que moins courante que **declare**). Elle permet de définir le type de données d'une variable, tout comme **declare** en Bash.

```
typeset OPTION NOM_VARIABLE=valeur
```

- La commande positionne, réinitialise ou affecte les variables selon différentes options :

- ◆ -L cadrage à gauche

```
$ var=MAVARIABLE; echo $var
MAVARIABLE
$ typeset -L4 var ; echo $var
MAVA
```

- ◆ -R cadrage à droite

```
$ var= MAVARIABLE ; echo $var
MAVARIABLE
typeset -R5 var ; echo $var
IABLE
```

- ◆ -Z cadrage à droite et remplissage par des 0 à gauche

```
$ typeset -Z5 var=$(ls | wc -l)
00005
```



# CHAPITRE 8 :

- ♦ conversion en majuscule **-u** ou en minuscule **-l**

```
$ var=MAVARIABLE ; print $var # Affiche MAVARIABLE
$ typeset -u var ; print $var # Affiche MAVARIABLE
$ typeset -l var ; print $var # Affiche mavARIABLE
```

- ♦ protection d'une variable en lecture uniquement **-r**

```
$ typeset -r var
$ var=bonjour # Affiche message : bash: var is read only
```

- ♦ **+r** annuler l'option (**-r**)

```
$ typeset -r var
$ var=bonjour # Affiche message : bash: var is read only
$ typeset +r var
$ var=bonjour; echo $var # Affiche bonjour
```

- ♦ De façon générale + annule l'option ou l'attribut affecté à une variable.

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Transmission des variables

- La transmission des variables d'environnement est unidirectionnelle : du processus père vers le(s) processus fils.
- Si un processus fils modifie la valeur d'une variable d'environnement, seule sa copie locale en sera affectée.

◆ Ex :

```
$ export a=coucou
$ bash
$ echo $a
coucou
$ a=bonsoir => modification de la variable d'environnement a dans le shell fils
$ echo $a
bonsoir
$ exit
$ echo $a
coucou => la valeur n'a pas été modifiée dans le shell père
$
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Transmission des variables

- Visualisation des variables disponibles
  - ◆ La commande **printenv** : affiche les valeurs des variables de l'environnement du processus en cours
- Supprimer une variable
  - ◆ La commande **unset** variable : supprime la variable de la liste des variables disponibles.

◆ Ex :

```
$printenv HOME
/home/mohammed
$ printenv PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
$ export VAR="Test"
$ printenv VAR # Vérifier la variable
Test
$ unset VAR
$ printenv VAR # Vérifier après suppression
(nothing)
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Substitutions des variables

- Effectuer un certain nombre de substitutions lors de l'affichage
  - ◆ La commande **printenv** : affiche les valeurs des variables de l'environnement du processus en cours
  - ◆ `$paramètre` => affiche la valeur courante de paramètre
  - ◆ `${paramètre}` => affiche la valeur courante de paramètre et autorise la concaténation
  - ◆ `${paramètre:-valeur}` => affiche la valeur courante de paramètre si elle est non nulle, sinon affiche Valeur
  - ◆ `${paramètre:=valeur}` => affiche la valeur courante de paramètre si elle est non nulle, sinon affiche valeur et affecte à paramètre valeur
  - ◆ `${paramètre:?valeur}` => affiche la valeur courante de paramètre si elle est non nulle, sinon on écrit valeur sur la sortie standard et on sort.

# CHAPITRE 8 :

---

## ■ Personnalisation de L'environnement

- ◆ **CDPATH**: liste des répertoires recherchés quand cd est exécuté avec un nom relatif comme argument,
- ◆ **DISPLAY**: l'écran sur lequel on travaille (par défaut :0.0), **ENV**: nom d'un fichier exécuté à chaque appel de bash,
- ◆ **GID**: le numéro du groupe actuel de l'utilisateur,
- ◆ **HOME**: répertoire de login,
- ◆ **HOST**: le nom de la machine sur laquelle on se trouve,
- ◆ **LOGNAME**: le nom de l'utilisateur,
- ◆ **MANPATH**: liste des répertoires où se trouvent les pages de manuel,
- ◆ **PATH**: liste des répertoire de recherche des commandes, le séparateur est «:»,
- ◆ **PWD**: le répertoire courant,
- ◆ **SHELL**: le shell par défaut,
- ◆ **TERM**: nom du type de terminal utilisé,

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

- Les paramètres de position sont des variables spéciales en Bash qui permettent d'accéder aux arguments passés à un script ou une fonction.
  - ◆ **\$0** Nom du script en cours d'exécution
  - ◆ **\$1, \$2, \$3, ...** Arguments passés au script (1er, 2e, 3e, etc.)
- L'assignation de valeurs à des paramètres de position s'effectue :
  - ◆ soit à l'aide de la commande interne **set**
  - ◆ soit lors de l'appel d'un fichier shell ou d'une fonction shell.
- La commande interne **set** affecte une valeur à un ou plusieurs paramètres de position en numérotant ses arguments suivant leur position. La numérotation commence à 1.
  - ◆ Syntaxe :
  
  - ◆ Ex :

## CHAPITRE 8 :

---

- Pour obtenir la valeur d'un paramètre de position, il suffit de placer le caractère **\$** devant son numéro. Ainsi, \$1 permet d'obtenir la valeur du premier paramètre de position, \$2 la valeur du deuxième et ainsi de suite.
  - ◆ Ex
- ◆ Tous les paramètres de position sont réinitialisés dès que la commande interne **set** est utilisée avec au moins un argument
- ◆ La commande interne **set --** rend indéfinie la valeur des paramètres de position préalablement initialisés.

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Paramètres spéciaux

- Un paramètre spécial est référencé par un caractère spécial. L'affectation d'une valeur à un paramètre spécial est effectuée par le shell (implicitement).
- Pour obtenir la valeur d'un paramètre spécial, il suffit de placer le caractère **\$** devant le caractère spécial qui le représente.
  - ◆ **\$#** donne le nombre d'arguments passés au script
  - ◆ **\${#var}** longueur de la variable "var"
  - ◆ **\$\*** liste complète des valeurs des paramètres
    - représente seul mot constitué de la valeur de tous les paramètres positionnels séparées par le premier caractère de la variable spéciale IFS.
    - Ceci signifie que **\$\*** est équivalent à "**\$1:\$2:...**", dans laquelle **:** est le premier caractère de la valeur de la variable IFS. Si IFS est nulle ou inexistante, les paramètres sont séparés par des espaces.
  - ◆ **\$0** le nom du script
  - ◆ **\$n, \${n}** valeur du nième argument du script,
  - ◆ **\$@** équivalent à **\$\***, mais **\$@** signifie: "**\$1**" "**\$2**" ... "**\${n}**", dont chaque paramètre se transforme en un mot distinct
  - ◆ **\$\$** le numéro de processus du script,



# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Exercice 3:

- Ecrire un programme shell **nbmots** qui demande à l'utilisateur de saisir une suite quelconque non vide de mots puis affiche le nombre de mots saisis .

- Ex :

```
$ nbmots
Entrez une suite de mots : un deux trois quatre cinq
=> 5 mots ont été saisis
```

- Correction :

```
Solution 1
#!/bin/bash
read -p "Entrez une suite de mots:" a
set $a
echo $#
```

```
Solution 2
#!/bin/bash
echo "Entrez une suite de mots"
read a
set $a
echo $#
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Chaines de caractères :

- Il existe plusieurs délimiteurs de chaînes de caractères
  - ◆ Apostrophe (simple quote) : *'texte'*
    - Le texte n'est pas du tout interprété
  - ◆ Guillemets (double quotes) : *"texte"*
    - Seuls les caractères \ , \$ et ` sont interprétés
    - Utilisé pour du texte qui contient des variables ou des caractères spéciaux
  - ◆ Anti-quotes : *`texte`*
    - Le texte est interprété comme une commande à exécuter. Le résultat de cette commande sera substitué
    - Utilisé dans le but d'exploiter le résultat d'une commande

## CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

### Chaines de caractères Exemple :

- Les simples quotes ' ' délimitent une chaîne de caractères. Même si cette chaîne contient des commandes ou des variables shell, celles-ci ne seront pas interprétées

```
$ variable="secret"
$ echo 'Mon mot de passe est $variable.'
Mon mot de passe est $variable.
```

- Les doubles quotes " " délimitent une chaîne de caractères, mais les noms de variable sont interprétés par le shell

```
$ variable="secret"
$ echo "Mon mot de passe est $variable."
Mon mot de passe est secret.
```

- Bash considère que les Anti-quotes `` délimitent une commande à exécuter. Les noms de variable et les commandes sont donc interprétés

```
$ echo `variable="connu"; echo "Mon mot de passe est $variable."`
Mon mot de passe est connu.
```

- echo `ls` cette commande affiche le contenu du répertoire courant à l'écran

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Opérations et opérateurs arithmétiques :

- La commande **expr** évalue les arguments comme une expression et le résultat est envoyé sur la sortie standard.

**expr arguments ...**
- « expr », comme toutes les commandes UNIX, doit avoir ses arguments séparés par des espaces.
- La première utilisation de « expr » concerne les opérations arithmétiques simples.
  - Les opérateurs +, -, \* et / correspondent respectivement à l'addition, à la soustraction, à la multiplication et à la division.
  - La seconde utilisation de la commande « expr » concerne la comparaison et la gestion des chaînes de caractères grâce à l'opérateur « : ».

```
#!/bin/bash
a=3 ; b=2
email="test@example.com"
nom="LinuxShell"
echo "Résultat de 3 + 5 : $(expr 3 + 5)"
echo "Résultat de a * b : $(expr $a * $b)"
position=$(expr index "$email" @)
echo "Position de '@' dans l'email : $position"
echo "Longueur de nom : ${#nom}"
echo "Les 5 premiers caractères de nom : ${nom:0:5}"
```

## Opérations et opérateurs arithmétiques :

### Expr et traitements des chaînes de caractères

- `expr exp1 \| exp2` renvoie `exp2` si l'expression n'est pas nulle, sinon `exp1`
- `expr exp1 \& exp2` renvoie `exp1` si l'expression n'est pas nulle, sinon `exp2`
- `expr exp1 : exp2` comparaison des deux arguments (renvoie le nombre de caractères en commun)
- `expr length "exp"` retourne le nombre de caractères de la chaîne `exp`, mais la syntaxe `${#exp}` est plus rapide et plus portable en Bash.
- `expr substr "exp" n1 n2` renvoie une sous-chaîne de `"exp"` commençant à la position `n1` et de longueur `n2`, mais la syntaxe `${exp:n1:n2}` est plus lisible et évite les problèmes d'ancienne syntaxe.
- `expr index "exp" "car"` retourne la position du caractère `car` dans la chaîne `exp`

## CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

---

- La commande permet de créer un tableau associatif où les clés peuvent être des chaînes de caractères au lieu d'être uniquement des entiers..

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Structure de contrôle « if »

- Permet d'effectuer une exécution conditionnelle

```
if [expression]
then
 # commandes à exécuter si la cond. est vraie
else
 # commandes à exécuter si la cond. est vraie
fi
```

- L'expression est constituée d'opérateurs
  - ◆ Liste des opérateurs numériques
    - **-eq** : Egalité (Equals)
    - **-ne** : Non égalité (Non Equals)
    - **-lt** : Infériorité (Less than)
    - **-le** : Infériorité ou égalité (Less or equals)
    - **-gt** : Supériorité (Greater then)
    - **-ge** : Supériorité ou égalité (Greater equals)

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Structure de contrôle « if »

- ◆ Liste des opérateurs sur chaîne de caractère
  - **-z** : Chaîne vide
  - **-n** : Chaîne non vide
  - **=** : Egalité de chaîne
  - **!=** : Non égalité de chaîne
- ◆ Liste des opérateurs sur fichiers
  - **-L** : Lien symbolique
  - **-d , -f** : Répertoire, Fichier
  - **-s** : Fichier non vide
  - **-r, -w, -x** : Droits qui s'appliquent (Lecteur, Ecriture, Exécution)
  - **-nt** : Plus récent (Newer than)
  - **-ot** : Plus vieux (Older than)



# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Structure de contrôle « if »

### ◆ Liste des opérateurs logiques

- **!** : Négation
- **-a** : Et (And)
- **-o** : Ou (Or)

### ■ Exemples

```
Teste si le paramètre $1 est égal à 2
if [$1 -eq $2]
then
 # Commande
fi
```

```
Teste si le paramètre $1 est un fichier OU un répertoire
if [-d $1 -o -f $2]
then
 # Commande
fi
```

```
Teste si le fichier existe
if [! -f "/etc/toto.conf"]
then
 # Commande exécutée si le fichier n'existe pas
fi
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Exercices

- Tester si le paramètre 2 est un fichier

```
if [-f $2]; then ; fi
```

- Tester si le paramètre 1 égale à toto ET paramètre 2 supérieur à 3

```
if [$1="toto" -a $2 -gt 3]; then ; fi
```

- Tester si le paramètre 1 n'est pas un répertoire ET s'il a le droit d'écriture

```
if [! -d $1 -a -w $1]; then ; fi
```

- Script capable de créer le répertoire passé en argument et de vérifier que sa création n'a pas provoqué d'erreurs

```
if [! -d $1]; then
 mkdir $1
 if [$? -ne 0]; then
 echo "Erreur lors de la création de $1«
 exit 1
 fi
fi
```

# CHAPITRE 8 :

---

## **case :**

- La commande **case esac** permet de vérifier le contenu d'une variable ou d'un résultat de manière multiple.
- **Expr** est soit un simple texte, soit composé de caractères spéciaux. Chaque bloc de commandes lié à **Expr** doit se terminer par deux points-virgules.
- Dès que le **Expr** est vérifié, le bloc de commandes correspondant est exécuté.
- \*) L'étoile en dernière position (chaîne variable) est l'action par défaut si aucun critère n'est vérifié.

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Select :

- Permet d'avoir un menu pour choisir la réponse

```
select var in selection1 selection2 ... selectionN
do
 instruction(s)
done
```

- Chaque sélection est affichée précédée par son numéro d'ordre
- La réponse est stockée dans une variable d'environnement **REPLY**
- La variable var contient le libellé du choix et REPLY l'indice de ce dernier.
- \$PS1 Le prompt principal du shell par défaut '\$'.
- \$PS2 Le prompt du shell utilisé quand une saisi va se faire par défaut '>'.
- L'utilisateur peut changer le prompt ou définir un nouveau avec \$PS3

```
echo "Sélectionnez votre système d'exploitation préféré :"
```

```
select systeme in "Windows" "Linux" "BSD" "Mac OS" "MS DOS"
```

```
do
```

```
 if [[-n "$systeme"]]; then # vérifie si une sélection valide a été faite
```

```
 echo "Votre système favori est $systeme."
```

```
 break # Sort de la boucle après un choix valide
```

```
 else
```

```
 echo "Option invalide, veuillez choisir un nombre correspondant à un système."
```

```
 fi
```

```
done
```

# CHAPITRE 8 :

---

- La commande `while` permet une boucle conditionnelle « tant que ». Tant que la condition est réalisée, le bloc de commande est exécuté. On sort si la condition n'est plus valable

- ou

- Ex .

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Boucles :

### La boucle until : jusqu'à ...

- La commande until permet une boucle conditionnelle « jusqu'à ». Dès que la condition est réalisée, on sort de la boucle.

```
until condition
do
 commandes
done
```

ou

```
until
do
 bloc d'instructions formant la condition
 commandes
done
```

- Ex .

```
#!/bin/bash

i=1 # Initialisation

until [$i -gt 6]; do # Tant que i <= 6
 echo "Welcome $i times."
 i=$((i+1)) # Incrémentation
done
```

```
#!/bin/bash

continuer="o" # Initialisation
Tant que l'utilisateur ne tape pas "n"
until ["$continuer" = "n"]; do
 echo "Voulez-vous recommencer ? (o/n)"
 read continuer
done

echo "Programme terminé."
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Boucles :

### La boucle for :

- L'itération for possède plusieurs syntaxes dont les deux plus générales sont :

```
for var
do
 suite_de_commandes
done
```

ou

```
for var in liste_mots
do
 suite_de_commandes
done
```

Lorsque cette syntaxe est utilisée, la variable var prend successivement la valeur de chaque paramètre de position initialisé.

La variable var prend successivement la valeur de chaque mot de liste\_mots .

```
programme for_arg

#!/bin/bash
for i
do
 echo $i
 echo "Passage a l'argument suivant ..."
done
```

```
#!/bin/bash
for a in toto tata
do
 echo $a
done
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Boucles :

### La boucle for :

- Bash a introduit une nouvelle structure for adaptée aux traitements des expressions arithmétiques, itération issue du langage C. Elle fonctionne comme cette dernière.

```
for ((expr_arith1 ; expr_arith2 ; expr_arith3))
do
 suite_cmd
done
```

- expr\_arith1 : est l'expression arithmétique d'initialisation.
- expr\_arith2 : est la condition d'arrêt de l'itération.
- expr\_arith3 : est l'expression arithmétique qui fixe le pas d'incrément ou de décrémentation.

```
#!/bin/bash

for ((x=0; x<5; x++)); do
 echo $((x * 2))
done
```





# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Fonctions

- Le shell bash propose plusieurs syntaxes pour définir une fonction:

```
function nom_fct
{
 suite_de_commandes
}
```

```
nom_fct ()
{
 commande ...
}
```

```
function nom_fct
{
 suite_de_commandes ;
}
```

- Pour appeler une fonction, il suffit de mentionner son nom.

```
nom_fct
```

- Les noms et corps de toutes les fonctions définies sont affichés à l'aide de la commande :

```
declare - f
```

- Pour afficher le nom et corps d'une ou plusieurs fonctions :

```
declare - f nom_fct
```

- Suppression d'une fonction :

```
unset - f nom_fct
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Retour de fonction :

- On peut définir une fonction qui retourne une valeur comme :

```
#!/bin/bash

Fonction qui teste si un nombre est pair ou impair
pair_impair() {
 if (($1 % 2 == 0)); then
 return 0 # 0 signifie vrai (pair)
 else
 return 1 # 1 signifie faux (impair)
 fi
}

Parcours des arguments passés au script
for nb in "$@"; do
 if pair_impair "$nb"; then
 echo "$nb est pair"
 else
 echo "$nb est impair"
 fi
done
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Renvoi d'une valeur par une fonction :

```
#!/bin/bash

Fonction qui retourne une somme
additionner() {
 echo $(($1 + $2))
}

Récupération du résultat dans une variable
resultat=$(additionner 5 7)

echo "Résultat de l'addition : $resultat"
```

# CHAPITRE 8 : Langage d'automatisation et de gestion du système SHELL

## Commandes break et continue

- La commande **break** permet d'interrompre une boucle. Dans ce cas le script continue après la commande **done**. Elle peut prendre un argument numérique indiquant le nombre de boucles à sauter, dans le cadre de boucles imbriquées (rapidement illisible).
- La commande **continue** permet de relancer une boucle et d'effectuer un nouveau passage. Elle peut prendre un argument numérique indiquant le nombre de boucles à relancer (on remonte de n boucles). Le script redémarre à la commande **do**.

```
#!/bin/bash

for ((i=1; i<=5; i++)); do
 echo "Tour n° $i"
 if ["$i" -eq 3]; then
 echo "Interruption avec break."
 break # Stoppe la boucle
 fi
done

echo "La boucle est terminée."
```

```
#!/bin/bash

for ((i=1; i<=5; i++)); do
 if ["$i" -eq 3]; then
 echo "saute l'itération n 3 avec continue."
 continue # Passe à l'itération suivante
 fi
 echo "Tour n° $i"
done
```